

simulation

November 10, 2018

1 Simulation

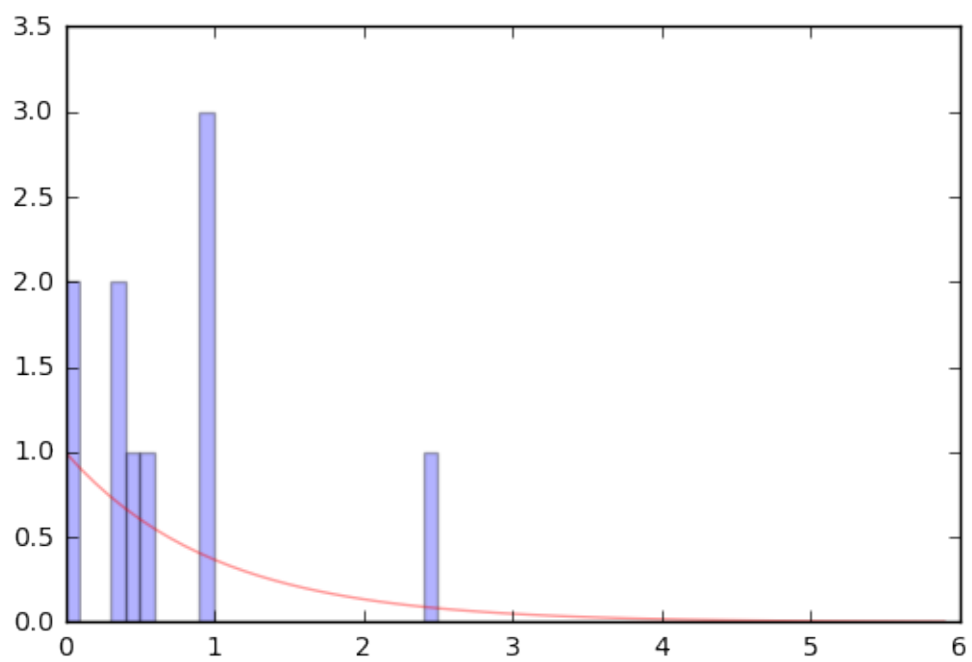
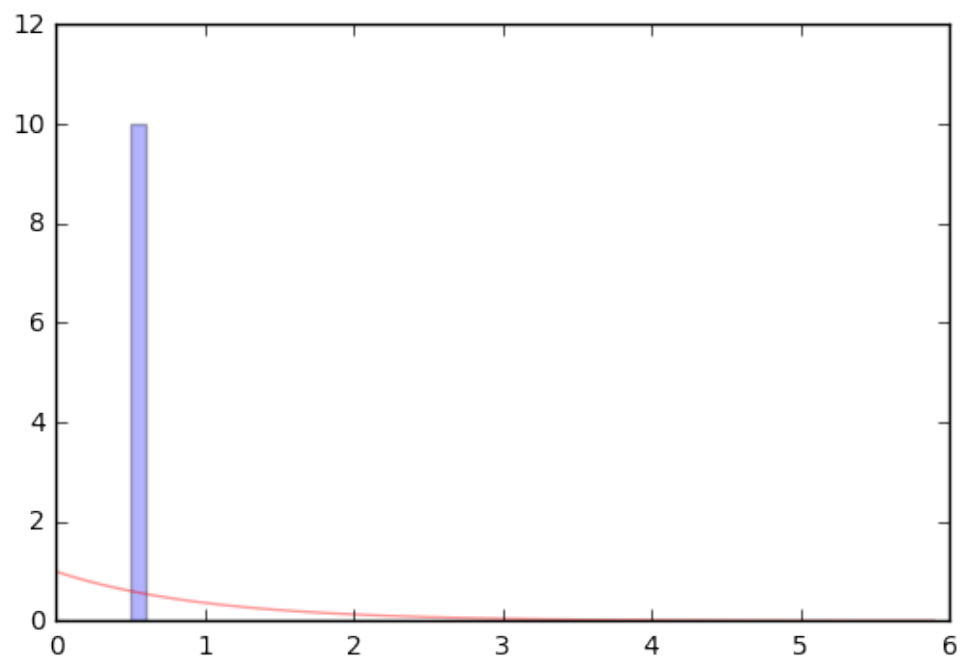
1.1 Construction de variables aléatoires de loi exponentielle

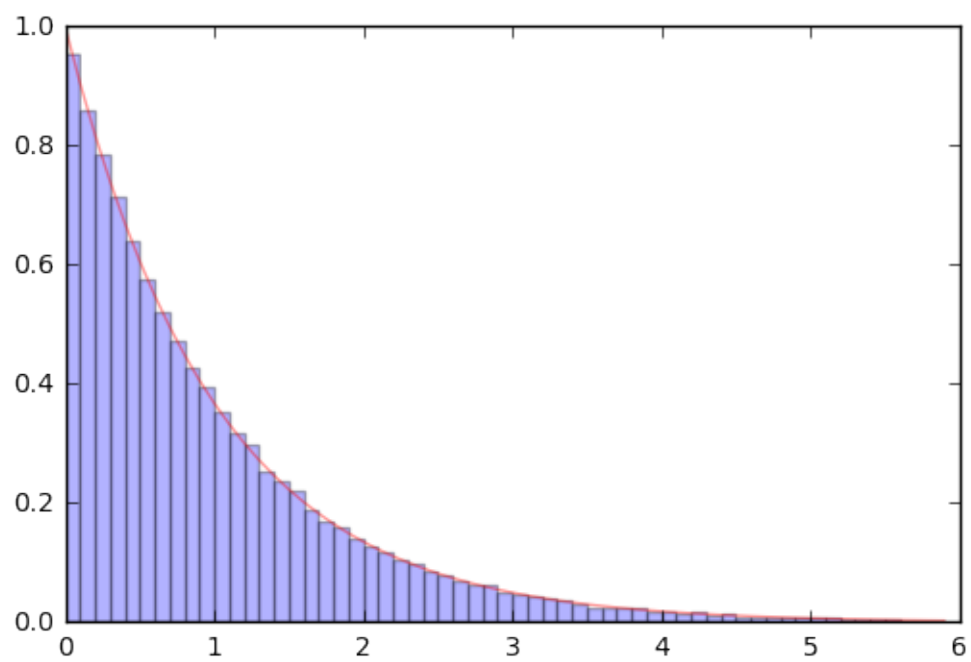
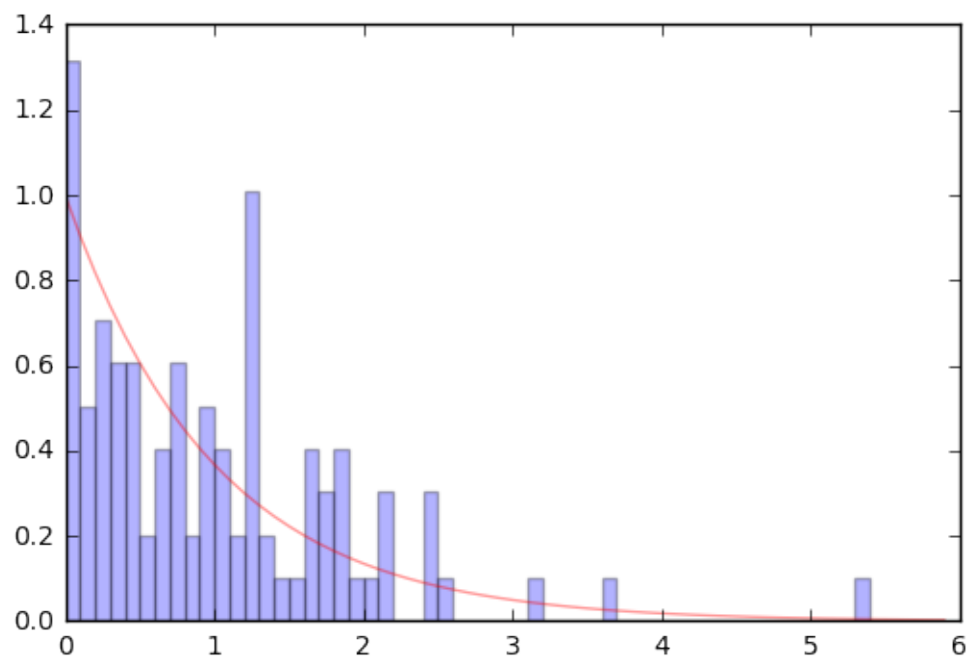
$$X = -\frac{1}{\lambda} \ln U$$

où U est une uniforme sur $[0, 1]$

```
In [2]: %matplotlib inline
        from numpy import *
        import numpy as np
        import scipy
        from pylab import *
        import pylab as pylab

        @interact
        def _(n = (0..6)) :
            pylab.clf()
            tableau=-log(np.random.rand(10**n,1))
            m1=pylab.hist(tableau,bins=srange(0, 6, 0.1),normed=1,alpha=0.3)
            pylab.plot(m1[1],exp(-m1[1]),color='red',alpha=0.4)
            pylab.savefig('m1')
```

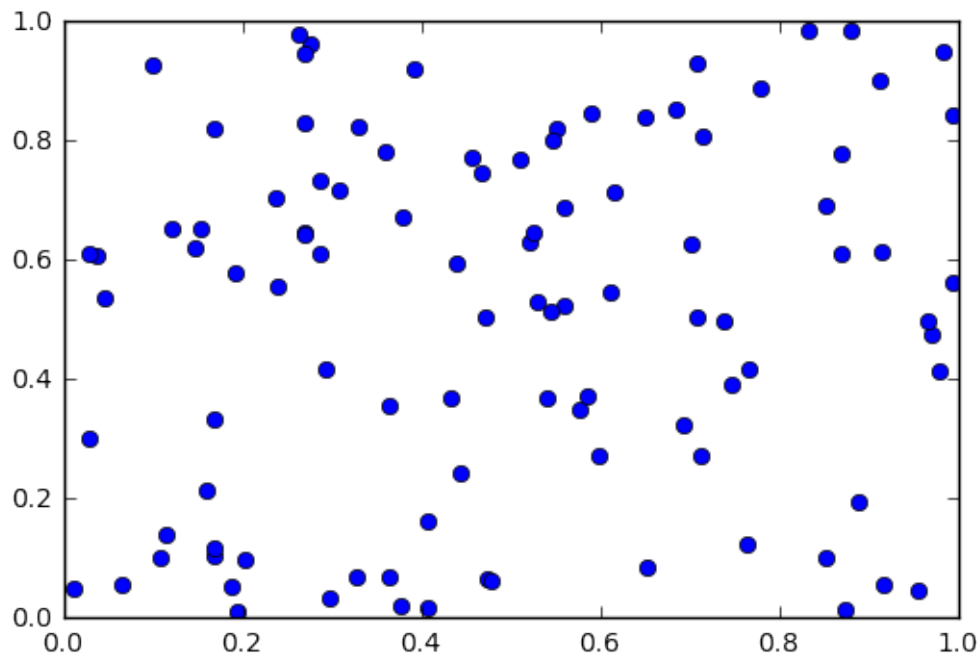




1.2 Tirages de point uniformément répartis dans le carré unité

On notera les “amalgames” qui semblent contredire l’idée d’uniformité mais qui sont en fait à l’indépendance des tirages

```
In [4]: pylab.clf()
        pylab.plot(np.random.rand(100), np.random.rand(100), 'o')
        pylab.savefig('cloud')
```



1.3 Estimation de pi

On tire des points (U_i, V_i) uniformément dans le carré $[0, 1] \times [0, 1]$. D’après la loi forte des grands nombres

$$\frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{U_i^2 + V_i^2 \leq 1\}} \longrightarrow \frac{\pi}{4}$$

```
In [6]: @interact
        def _(n = (0..8)) :
            N=10**n
            u=2*np.random.rand(N)-1
            v=2*np.random.rand(N)-1
            frequence=sum(u^2+v^2<=1)*1.0/N
            variance=frequence*(1-frequence)
            print "estimation de pi ", 4*frequence
            print "\nintervalle de confiance à 95% ", 4*(frequence-1.96*sqrt(variance))
```

1.4 Simulation de la file M/M/1

Intensité = $\rho < 1$

Temps moyen de service = 1

On calcule l'histogramme empirique de la distribution du nombre de clients dans le système

```
In [10]: rho=0.5
horizon=10000
temps=0
N=0
K=int(np.trunc(np.log(0.001)/np.log(rho)))+1
proba_stat=np.zeros(K+1)
nb_moyen_clients=0

def transitions(N):
    if (N==0):
        return([rho,[1],[1]])
    else:
        return([rho+1,[N-1,N+1],[1/(rho+1),rho/(rho+1)]])

while(temps<= horizon):
    a = transitions(N)
    # print N,a
    duree=np.random.exponential(1/a[0])
    temps += duree
    proba_stat[min(N,K)]+= duree
    N=np.random.choice(a[1],p=a[2])
    # print N

proba_stat =proba_stat/horizon
pylab.clf()
pylab.bar(np.arange(K+1),proba_stat)
pylab.plot(np.arange(K+1),(1-rho)*rho**np.arange(K+1),color='red')
pylab.savefig('geometric')
```

